



Estrutura de Dados não Lineares

Prof. Thiago Koman Schmidt



Planejamento da aula

- Ordem de Crescimento
- Big-O, Big- Ω , Big- Θ
- Comparações entre notações
- Interpretação em gráficos e tabelas
- Classificação dos algoritmos segundo o crescimento
- Exemplos práticos de cada classe
- Impacto da classe no desempenho computacional

Por que se preocupar com crescimento?

Algoritmos podem resolver o mesmo problema, mas com custos diferentes.

- Entender a eficiência algorítmica evita gargalos de desempenho.
- Avaliamos como tempo e espaço crescem com a entrada n .
- O que funciona para $n = 100$ pode falhar em $n = 1.000.000$.
- A complexidade determina escalabilidade e viabilidade de soluções.

Exemplo real:

- Banco de dados com 10 milhões de registros:
- Algoritmo $O(n)$ → viável
- Algoritmo $O(n^2)$ → pode travar

O que é Ordem de Crescimento?

É a forma como o **tempo de execução** ou o **uso de memória** de um algoritmo cresce com o tamanho da entrada n .



Desafio

Desafio

Escreva uma função em C++ chamada **somar** que receba um número inteiro **n** como parâmetro e retorne a soma de todos os números inteiros de 1 até **n**.

Requisitos:

O programa principal (main) deve solicitar ao usuário o valor de **n**, chamar a função e exibir o resultado.

Crescimento em Códigos

```
// O(1)
int soma_constante(int n) {
    return n * (n + 1) / 2;
}

// O(n)
int soma_linear(int n) {
    int soma = 0;
    for (int i = 1; i <= n; i++) {
        soma += i;
    }
    return soma;
}
```

*Mesmo resultado, mas eficiência muito diferente com grandes entradas.

Principais Classes de Complexidade

Classe	Notação	Exemplo	Desempenho para n grande
Constante	$O(1)$	Acesso direto em vetor	Excelente
Logarítmica	$O(\log n)$	Busca binária	Muito boa
Linear	$O(n)$	Busca linear em vetor	Boa
Linearítmica	$O(n \log n)$	Merge Sort, Quick Sort médio	Razoável
Quadrática	$O(n^2)$	Bubble Sort, Selection Sort	Ruim
Cúbica	$O(n^3)$	Algoritmos de força bruta	Muito ruim
Exponencial	$O(2^n)$	Problemas de força bruta	Impraticável
Fatorial	$O(n!)$	Algoritmos de permutação total	Extremamente impraticável

Big-O Complexity Chart

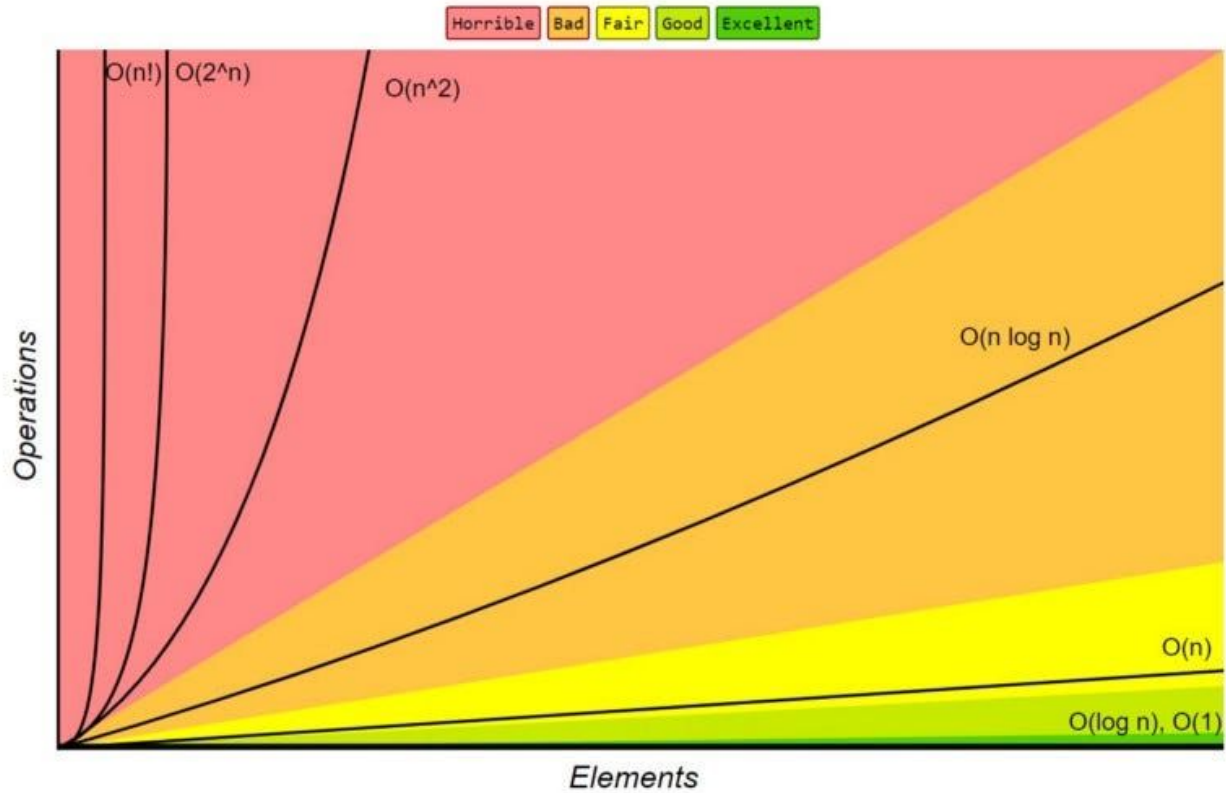


Tabela Comparativa de Crescimento

n	$O(1)$	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$	$O(2^n)$
10	1	~3	10	~33	100	1024
100	1	~6.6	100	~660	10.000	1.27e30
1000	1	~10	1000	~10.000	1.000.000	1e300



Notações Assintóticas

Introdução às Notações Assintóticas

Notação	Nome	Descrição
$O(n)$	Big-0	Limite superior (pior caso)
$\Omega(n)$	Big-Omega	Limite inferior (melhor caso)
$\Theta(n)$	Big-Theta	Crescimento exato (médio ou geral)

- Notações são importantes para análise teórica de algoritmos
- Ignoram constantes e implementações específicas

Big-O (O grande)

Define o limite superior do crescimento do algoritmo.

- Descreve o pior caso.

```
bool busca(int v[], int n, int x) {  
    for (int i = 0; i < n; i++) {  
        if (v[i] == x) return true;  
    }  
    return false;  
}  
  
// Pior caso: x não está → O(n)
```

Big-O (O grande)

- **Descreve o pior caso.**

- $v = [0, 1, 2, 3, 4, 5, 6]$
- $n = 7$
- $x = 8$

```
bool busca(int v[], int n, int x) {  
    for (int i = 0; i < n; i++) {  
        if (v[i] == x) return true;  
    }  
    return false;  
}  
  
// Pior caso: x não está →  $O(n)$ 
```

Big-Ω (Ômega)

Define o limite inferior do crescimento.

- Descreve o melhor caso.

```
bool busca(int v[], int n, int x) {  
    for (int i = 0; i < n; i++) {  
        if (v[i] == x) return true;  
    }  
    return false;  
}
```

// Melhor caso: x é o primeiro elemento → $\Omega(1)$

Big-Ω (Ômega)

- **Descreve o melhor caso.**

- $v = [0, 1, 2, 3, 4, 5, 6]$
- $n = 7$
- $x = 0$

```
bool busca(int v[], int n, int x) {  
    for (int i = 0; i < n; i++) {  
        if (v[i] == x) return true;  
    }  
    return false;  
}
```

// Melhor caso: x é o primeiro elemento $\rightarrow \Omega(1)$

Big- Θ (Teta)

Define o crescimento exato do algoritmo.

- Se o comportamento no melhor e pior caso é o mesmo.

```
void imprime(int v[], int n) {  
    for (int i = 0; i < n; i++) {  
        cout << v[i] << endl;  
    }  
}  
  
// Sempre imprime n vezes  $\rightarrow \theta(n)$ 
```

Comparando as Notações

Notação	Significado	Exemplo
$O(n)$	No máximo n passos	Pior caso
$\Omega(n)$	No mínimo n passos	Melhor caso
$\theta(n)$	Exatamente n passos	Casos iguais no melhor e pior



Desafio

Desafio

Utilize o algoritmo Bubble Sort para ordenar um vetor de números inteiros.

Implementação:

Escreva o código em C++ que implemente o Bubble Sort para ordenar os dados fornecidos.

Análise de desempenho:

Após a implementação, analise o algoritmo considerando os seguintes aspectos:

- **Melhor caso:** Explique e exemplifique quando o algoritmo apresenta seu melhor desempenho.
- **Pior caso:** Explique e exemplifique a situação em que o algoritmo tem o pior desempenho.
- **Complexidade assintótica:** Descreva as notações de tempo de execução para o Bubble Sort usando as notações Big O (O), Big Omega (Ω) e Big Theta (Θ), relacionando-as ao melhor, pior e caso médio.

Desafio - Bubble Sort

```
void bubbleSort(int arr[], int n) {  
    bool trocou;  
    for (int i = 0; i < n - 1; i++) {  
        trocou = false;  
        for (int j = 0; j < n - 1 - i; j++) {  
            if (arr[j] > arr[j + 1]) {  
                // Troca os elementos  
                int temp = arr[j];  
                arr[j] = arr[j + 1];  
                arr[j + 1] = temp;  
                trocou = true;  
            }  
        }  
        // Se não houve troca, o array já está ordenado  
        if (!trocou)  
            break;  
    }  
}
```



Classes de Algoritmos

Introdução ao Crescimento Assintótico

- O crescimento do tempo de execução de um algoritmo depende do tamanho da entrada (n).
- Classes de complexidade nos ajudam a prever a escalabilidade.
- Notamos essas classes com notações como: $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, etc.

Principais Classes de Complexidade

Classe	Notação	Exemplo	Desempenho para n grande
Constante	$O(1)$	Acesso direto em vetor	Excelente
Logarítmica	$O(\log n)$	Busca binária	Muito boa
Linear	$O(n)$	Busca linear em vetor	Boa
Linearítmica	$O(n \log n)$	Merge Sort, Quick Sort médio	Razoável
Quadrática	$O(n^2)$	Bubble Sort, Selection Sort	Ruim
Cúbica	$O(n^3)$	Algoritmos de força bruta	Muito ruim
Exponencial	$O(2^n)$	Problemas de força bruta	Impraticável
Fatorial	$O(n!)$	Algoritmos de permutação total	Extremamente impraticável

Crescimento Logarítmico - $O(\log n)$

Aumenta lentamente conforme n cresce.

Exemplo clássico: Busca Binária

```
int buscaBinaria(int arr[], int n, int x) {  
    int esquerda = 0, direita = n - 1;  
    while (esquerda <= direita) {  
        int meio = (esquerda + direita) / 2;  
        if (arr[meio] == x) return meio;  
        if (arr[meio] < x) esquerda = meio + 1;  
        else direita = meio - 1;  
    }  
    return -1;  
}
```

****Escala bem para grandes volumes de dados ordenados.**

Crescimento Linear - $O(n)$

Cresce proporcionalmente ao tamanho da entrada.

Exemplo: Busca Linear

```
int buscaLinear(int arr[], int n, int x) {  
    for (int i = 0; i < n; i++) {  
        if (arr[i] == x) return i;  
    }  
    return -1;  
}
```

******Bom para listas pequenas ou não ordenadas.

Crescimento Linearítmico - $O(n \log n)$

Combina crescimento linear com logarítmico.

Exemplo: Merge Sort, Quick Sort (caso médio)

```
void mergeSort(int arr[], int l, int r) {  
    if (l < r) {  
        int m = l + (r - l) / 2;  
        mergeSort(arr, l, m);  
        mergeSort(arr, m + 1, r);  
        merge(arr, l, m, r);  
    }  
}
```

******Muito eficiente para ordenação de grandes conjuntos.

Crescimento Quadrático - $O(n^2)$

Tempo cresce proporcional ao quadrado do tamanho da entrada.

Exemplo: Bubble Sort, Insertion Sort

```
void bubbleSort(int arr[], int n) {  
    for (int i = 0; i < n-1; i++) {  
        for (int j = 0; j < n-i-1; j++) {  
            if (arr[j] > arr[j+1]) {  
                swap(arr[j], arr[j+1]);  
            }  
        }  
    }  
}
```

****Ineficiente para grandes entradas.**

Crescimento Cúbico ($O(n^3)$)

Complexidade ainda mais custosa que $O(n^2)$.

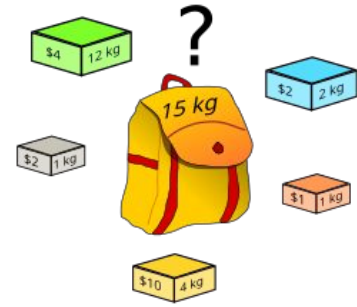
Exemplo: Algoritmo de Floyd-Warshall (caminhos múltiplos entre todos os pares de vértices).

```
for (int k = 0; k < n; k++) {  
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < n; j++) {  
            dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);  
        }  
    }  
}
```

Crescimento Exponencial - $O(2^n)$

Tempo dobra a cada nova entrada.

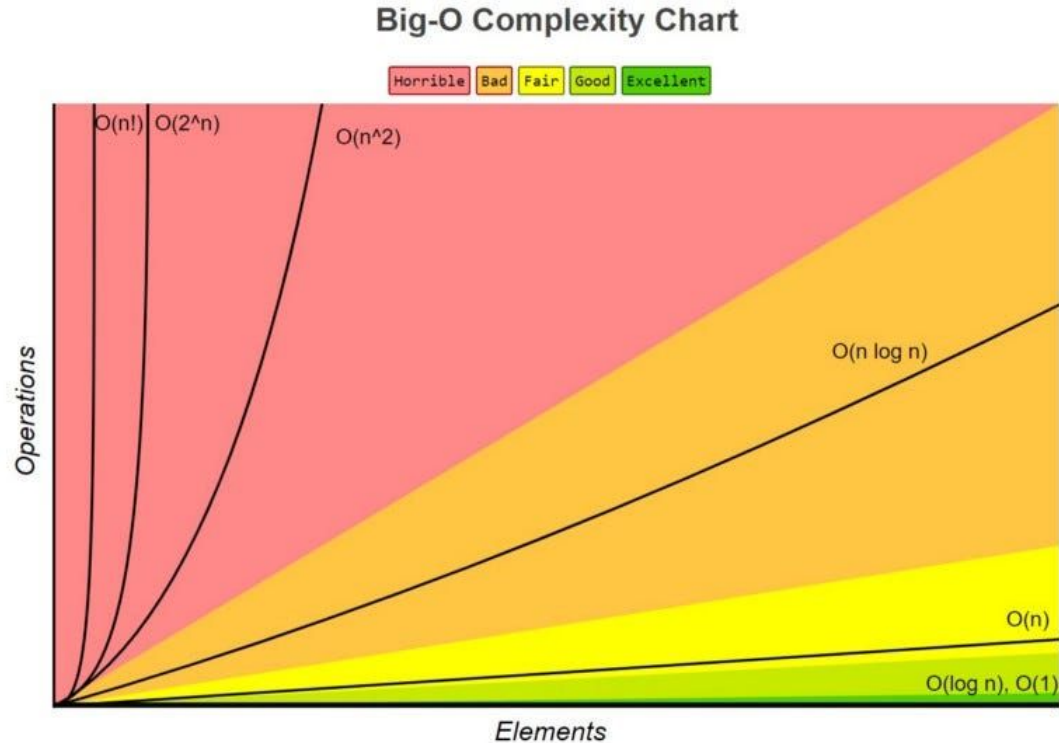
Exemplo: Força bruta para resolver o problema da Mochila, Subconjuntos, etc.



```
int fib(int n) {  
    if (n <= 1) return n;  
    return fib(n-1) + fib(n-2);  
}
```

****Praticamente inviável para entradas grandes.**

Comparativo Visual das Classes



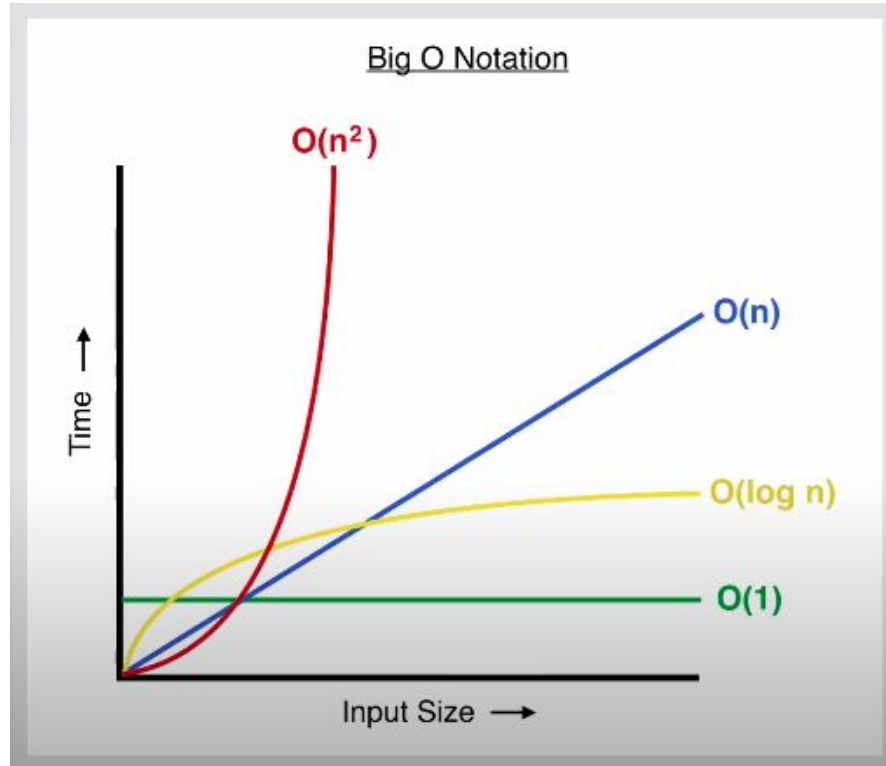


Impacto no desempenho

Impacto Prático no Desempenho

- Algoritmos com complexidade alta podem ser aceitáveis para pequenas entradas.
- Para aplicações com grandes dados (Big Data, IA), algoritmos eficientes são essenciais.
- Decisões de projeto devem considerar essas classes.
- Sempre que possível, ao refatorar um script, o desenvolvedor deve analisar a possibilidade de **baixar o nível de complexidade**, utilizando os conceitos de **classe de complexidade**, do algoritmo;

Impacto Prático no Desempenho





Atividade

Atividade 1

Implemente em C++ o seguinte algoritmo:

```
int exemplo(int n) {  
    int cont = 0;  
    for (int i = 0; i < n; i++) {           // Loop externo  
        for (int j = 0; j < n; j++) {       // Loop interno  
            cont++;  
        }  
    }  
    return cont;  
}
```

Tarefas:

1. Estime o número de operações em função de **n**.
2. Classifique a complexidade assintótica.
3. Compare a contagem real (*cont* impresso na tela) para **n = 10, 100, 1000**.

Atividade 2

Implemente um programa em C++ que:

Calcule e imprima o tempo de execução de três funções diferentes:

- **f1(n)**: percorre um vetor de tamanho n uma vez ($O(n)$).
- **f2(n)**: percorre o vetor duas vezes em laços aninhados ($O(n^2)$).
- **f3(n)**: percorre o vetor e aplica busca binária a cada elemento ($O(n \log n)$).

Tarefas:

1. Execute para $n = 10^3, 10^4, 10^5$.
2. Monte uma pequena tabela comparando os tempos.
3. Explique como os resultados confirmam ou não a teoria da ordem de crescimento.

Atividade 3

Imagine que você tem um problema de ordenação com 10^6 elementos.

Você dispõe de dois algoritmos já implementados em C++:

- **Algoritmo A:** Bubble Sort ($O(n^2)$)
- **Algoritmo B:** Merge Sort ($O(n \log n)$)

Tarefas:

1. Estime aproximadamente quantas comparações cada algoritmo realizaria.
2. Simule em código C++ ambos os algoritmos com vetores grandes e registre o tempo.
3. Discuta em quais situações práticas o algoritmo quadrático ainda poderia ser escolhido.

DÚVIDAS?

